



University
of Exeter

COM3031 Probabilistic Machine Learning: Week 9

Term 2, 2025-2026

Dr Zeyu Fu, Lecturer in Computer Vision
and Machine Learning

z.fu@exeter.ac.uk

In this week

Lectures

- Introduction to Reinforcement Learning
- Multi-armed Bandits
- Markov Decision Process
- Q Learning

Workshop

- Information Theory and Hidden Markov Models

Acknowledgements: some lecture notes were adapted from David Silver, Sargur N. Srihari, Sayak Paul, Thomas Sauerwald and Chathurangi Shyalika

Part I: Introduction to Reinforcement learning

From Hidden States to Decision Making

From Hidden Markov Models to Reinforcement Learning

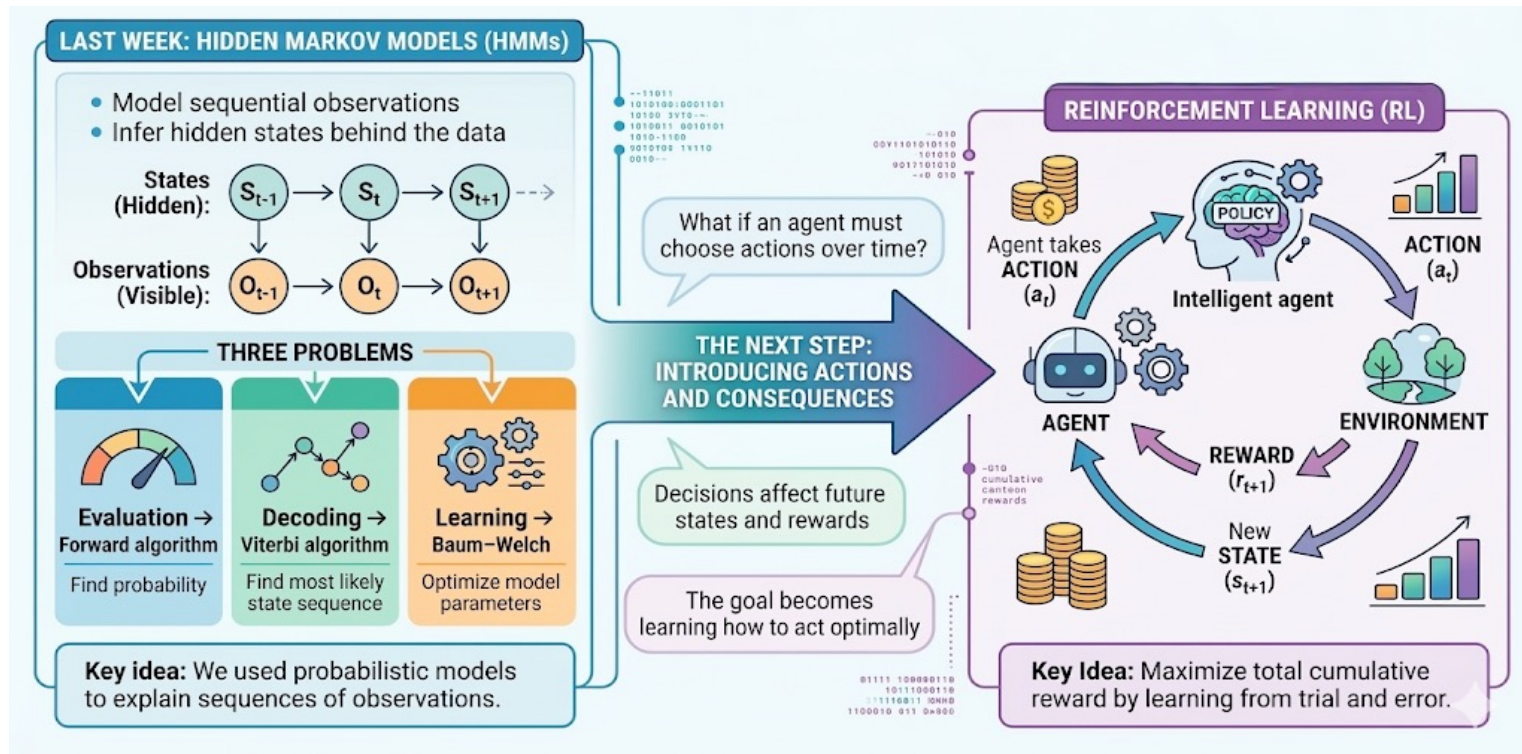


Figure generated using generative AI, adapted by the authors

Learning from Feedback

Different learning paradigms provide different levels of feedback.

- ▶ **“Pure” Reinforcement Learning (cherry)**
 - ▶ The machine predicts a scalar reward given once in a while.
 - ▶ **A few bits for some samples**
- ▶ **Supervised Learning (icing)**
 - ▶ The machine predicts a category or a few numbers for each input
 - ▶ Predicting human-supplied data
 - ▶ **10 → 10,000 bits per sample**
- ▶ **Self-Supervised Learning (cake génoise)**
 - ▶ The machine predicts any part of its input for any observed part.
 - ▶ Predicts future frames in videos
 - ▶ **Millions of bits per sample**



How can an agent learn good behaviour from sparse feedback?

Learning from Interaction

Core Idea

- An agent learns through interaction with an environment to choose actions that maximise long-term rewards.

Intuition — Training a Dog

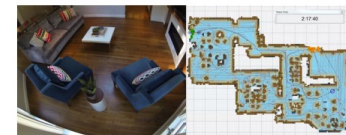
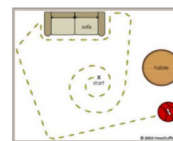
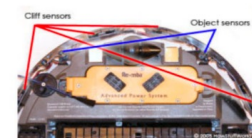
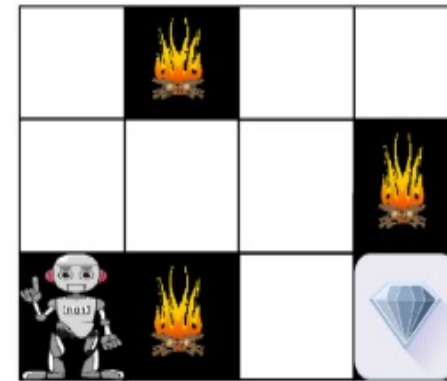
- Reward good behaviour
- The agent learns which actions produce reward

Simple Environment — Grid World

- Agent navigates a grid
- Goal: reach reward, avoid penalties
- Learns a **policy** through exploration

Real System — Robot Vacuum

- Explores rooms and avoids obstacles
- Learns to maximize cleaning efficiency



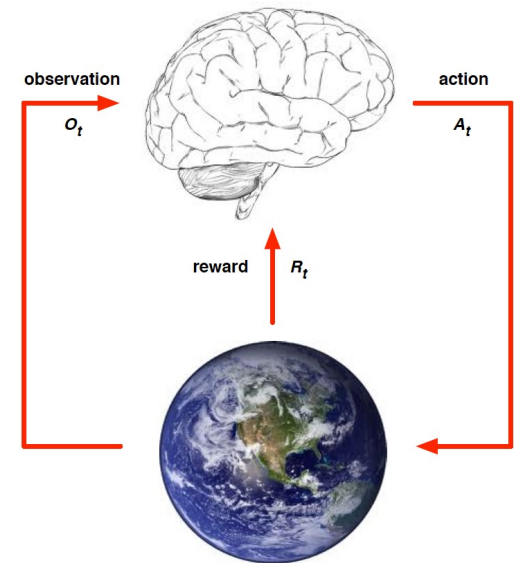
What Makes RL Different?

Reinforcement learning studies agents that learn by interacting with an environment.

- The agent learns through **trial and error**
- It receives **rewards instead of labelled answers**.

Key Characteristics of RL

- **No supervisor:** the agent does not see the correct action, only a reward signal.
- **Delayed feedback:** actions may influence rewards much later in time.
- **Sequential decisions:** The problem is not i.i.d. , past actions influence future states.
- **Actions change the data:** The agent's behaviour affects the observations it receives.



Applications of RL

Reinforcement Learning is used in many real-world decision problems.

Game playing

- Chess, Go, [Atari games](#) (e.g., AlphaGo)

Robotics

- Learning control policies for locomotion and manipulation

Recommendation systems

- Ad placement and personalized content

Autonomous systems

- Self-driving cars and drone navigation

Resource management

- Traffic control, energy systems, cloud computing

RL and other disciplines

- RL has a very close relationship with Psychology, Biology, Control Engineering, Economics, and Neuroscience.

Reinforcement Learning Framework

Core Idea: an agent interacts with an environment to learn how to act.

- The environment is initially unknown
- The agent interacts with the environment repeatedly
- Each interaction produces experience

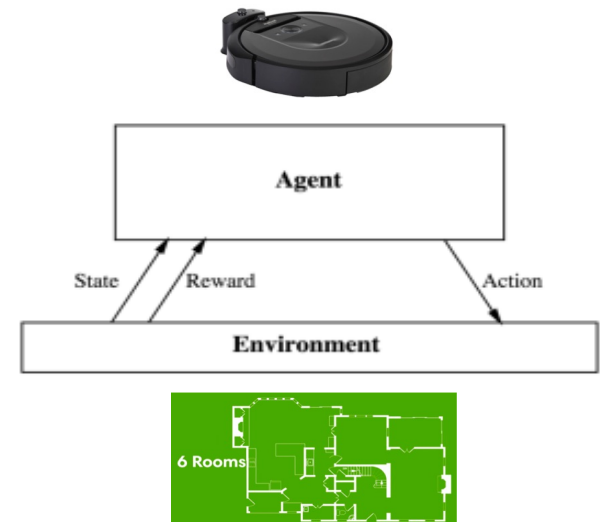
Interaction Loop

At each time step:

- The agent observes the state
- The agent chooses an action
- The environment returns a reward and the next state

Learning Objective

- The agent learns a policy (strategy) that chooses actions to maximise long-term reward.



RL Terminology

Key Reinforcement Learning Components

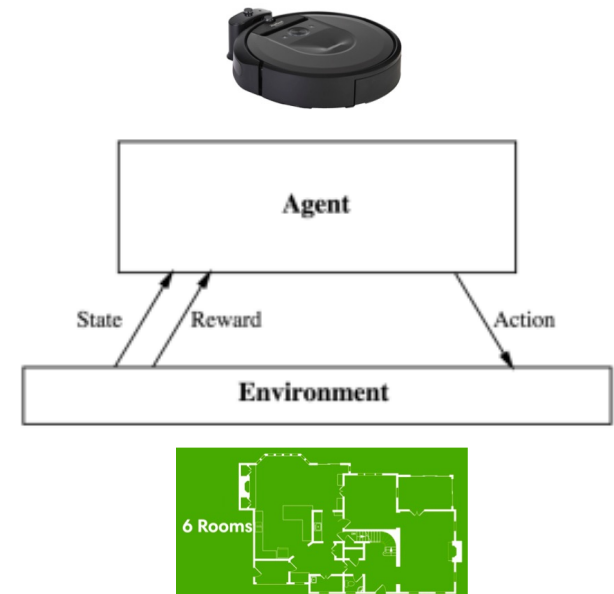
- **Action (a)** – possible moves that agent can take
- **State (s)** – current situation returned by the environment
- **Reward (r)** – Immediate return sent back from the environment to evaluate the action
- **Policy (π)** – Strategy that agent employs to determine next action based on the current state
- **Value (v)** – Expected long-term return with discount, as opposed to short-term reward

Interaction Loop

At each step:

$$s_t \rightarrow a_t \rightarrow r_{t+1}, s_{t+1}$$

- The agent learns a policy that maximizes long-term reward.



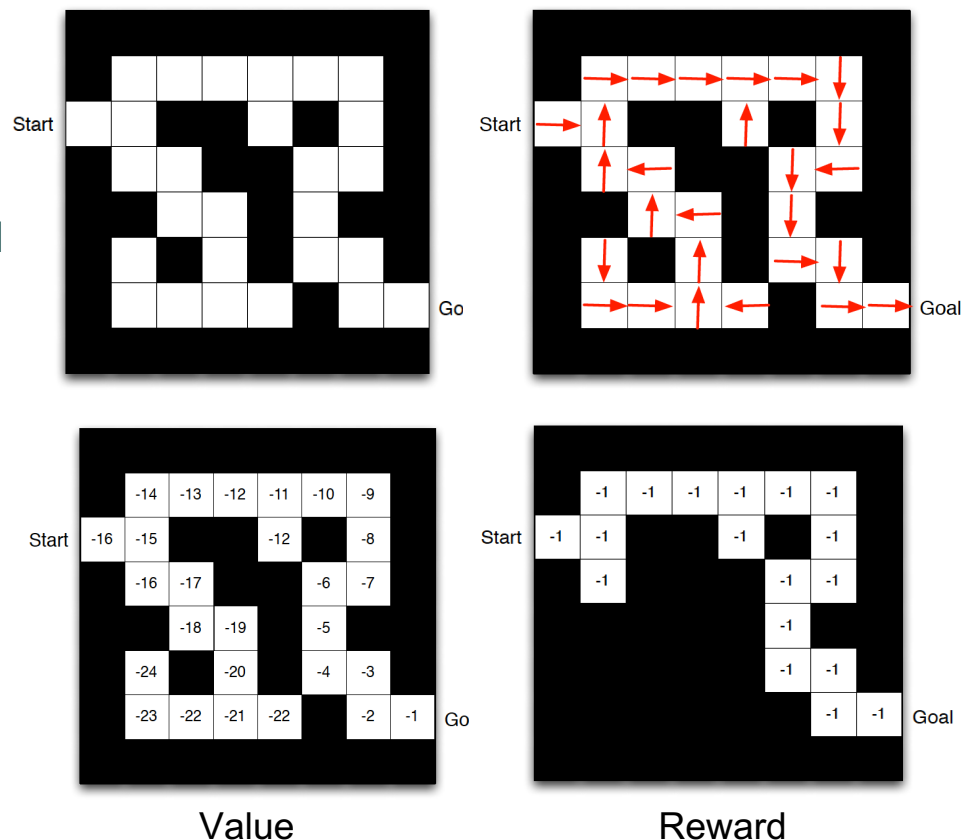
RL Terminology (Example)

Environment

- **States:** agent's location in the grid
- **Actions:** move North, East, South, West
- **Reward:** -1 per step until reaching the goal
- **Value:** numbers represent the expected long-term value of each state
- **Policy:** Arrows show the best action in each state

Learning Goal

- Learn a policy that moves the agent from Start to Goal efficiently.



Exploration vs Exploitation

Key questions: How should the agent decide which action to take?

- A reinforcement learning agent must balance two strategies:

Exploitation

- Choose the action that currently seems best and use the knowledge already learned
- Goal: maximise **immediate reward**
- Example: A robot vacuum follows the path it already knows and cleans efficiently.

Exploration

- Try new actions to gather more information
- Discover potentially **better strategies**
- Example: the robot vacuum tries a new route it has never used before.

Key Challenge

- Only exploiting → may miss better actions
- Only exploring → inefficient learning

Good RL balances exploration and exploitation

Exploration vs Exploitation (Examples)

The exploration–exploitation trade-off appears in many decision-making problems.

Restaurant Selection

- Exploit: Go to your favourite restaurant
- Explore: Try a new restaurant that might be better

Online Banner Advertisements

- Exploit: Show the most successful advert
- Explore: Show a different advert

Oil Exploration

- Exploit: Drill at the most promising known location
- Explore: Test a new location that might contain more oil

Game Playing

- Exploit: Play the move believed to be best
- Explore: Try a new move to discover stronger strategies

Part II: Multi-Armed Bandits

Multi-Armed Bandits

From Exploration vs Exploitation

- Imagine a casino with several slot machines (“arms”).
- Each machine gives rewards, but we do not know which machine is best.

Learning Task

At each time step t :

- The agent chooses an action (pulls an arm)

$$a_t \in \mathcal{A}$$

- The environment returns a reward

$$r_t \sim \mathcal{R}^{a_t}$$

- where $\mathcal{R}^a(r) = P(r|a)$ is an unknown reward distribution

Objective: After interacting for T steps, the goal is to maximize the total reward $\sum_{t=1}^T r_t$.

Key Challenge

The agent must decide whether to:

- **Exploit:** choose the arm that currently seems best
- **Explore:** try other arms to gather more information



Multi-Armed Bandits

Action Values

- Each action produces rewards drawn from a distribution.
- The expected reward of an action is called the action value:

$$Q(a) = \mathbb{E}[r \mid a]$$

- $Q(a)$ represents the average reward obtained when choosing action a .
- Rewards depend only on the selected action and are assumed to be drawn from the same distribution each time.

Optimal Action

- The best action is

$$a^* = \arg \max_a Q(a)$$

- with optimal value

$$V^* = Q(a^*)$$

However:

- The true action values $Q(a)$ are unknown and must be learned.

Multi-Armed Bandits

Measuring Performance: Regret

- If the agent selects action a_t , the **instant regret** is

$$l_t = V^* - Q(a_t)$$

- This measures how much reward is lost compared to the optimal action.
- Total regret after T steps:

$$L_T = \sum_{t=1}^T (V^* - Q(a_t))$$

Goal : Maximize cumulative reward $\Leftrightarrow$ Minimize total regret

Greedy Action Selection

- Estimate action values from observed rewards:

$$\widehat{Q}_t(a) = \frac{\text{sum of rewards from action } a}{\text{number of times action } a \text{ chosen}}$$

- Choose

$$a_t = \arg \max_a \widehat{Q}_t(a)$$

- Problem: Greedy may stop exploring too early and get stuck with a suboptimal action

Multi-Armed Bandits

Greedy Algorithm Example

- Consider a **3-armed bandit**: 🎰 Arm 1 🎰 Arm 2 🎰 Arm 3
- Initial action-value estimates: $Q_0(1) = Q_0(2) = Q_0(3) = 0$

t	Reward r_t	Estimated Values $Q_t(1), Q_t(2), Q_t(3)$	Chosen Action
1	0	(0, 0, 0)	1
2	1	(0, 0, 0)	2
3	1	(0, 1, 0)	2
4	0	(0, 1, 0)	2
5	1	(0, 2/3, 0)	2
6	0	(0, 3/4, 0)	2

- Suppose early rewards make Arm 2 appear best. The greedy algorithm will repeatedly select **Arm 2**.
- Arm **3 is never selected**, even though it might actually have the highest reward.
- Limitation: Greedy exploits too early and may miss better action

Multi-Armed Bandits

ε -Greedy Algorithm

Problem:

- the greedy algorithm always chooses the action with the highest estimated value.
- It may never explore other actions.

Idea

- ε -Greedy balances exploration and exploitation.
- Most of the time we exploit, but occasionally we explore.

Action Selection Rule

- At each step t :

$$a_t = \begin{cases} \arg \max_a \widehat{Q}_t(a) & \text{with probability } 1 - \varepsilon \\ \text{random action} & \text{with probability } \varepsilon \end{cases}$$

Interpretation

- Exploit the best known action most of the time
- Explore occasionally to discover better actions

Multi-Armed Bandits

Consider the same **3-armed bandit example** as before.

Initial estimates: $Q_0(1) = Q_0(2) = Q_0(3) = 0$

- With probability $1 - \varepsilon \rightarrow$ choose the greedy action
- With probability $\varepsilon \rightarrow$ choose a random action

t	Estimated Values $Q_t(1), Q_t(2), Q_t(3)$	Selected Action	Reward
1	(0,0,0)	1	0
2	(0,0,0)	2	1
3	(0,1,0)	1 (<i>explore</i>)	0
4	(0,1,0)	2	0
5	(0,0.5,0)	3 (<i>explore</i>)	0
6	(0,0.5,0)	2	0
7	(0,0.33,0)	2	0
8	(0,0.25,0)	1 (<i>explore</i>)	0
9	(0,0.25,0)	3 (<i>explore</i>)	1
10	(0,0.25,0.5)	3	1

- Early steps may include suboptimal actions due to exploration.
- However, all actions are eventually tried. This allows the agent to discover better arms.

Part III: Markov Decision Process

Why Multi-Armed Bandits Are Not Enough

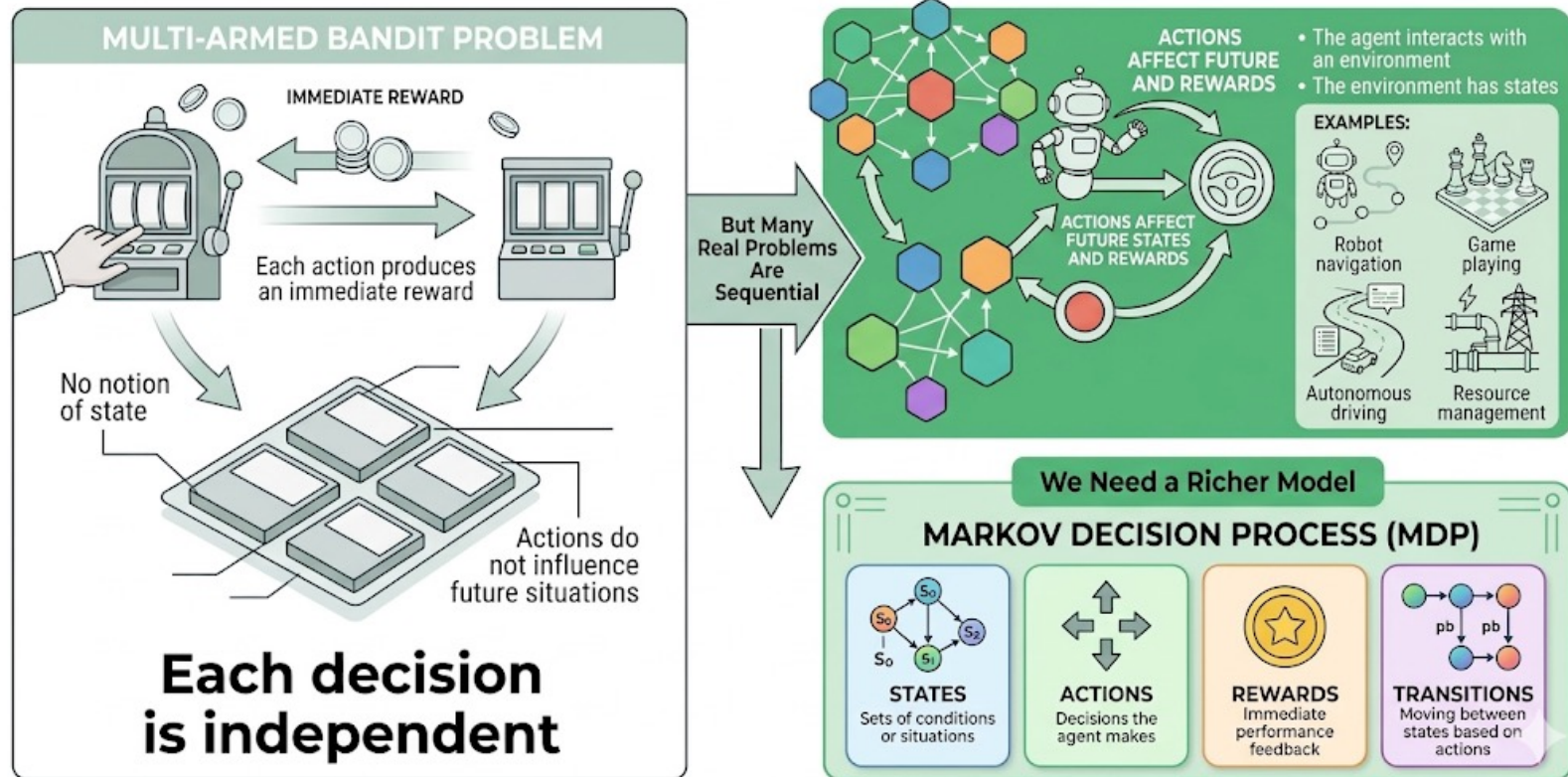


Figure generated using generative AI, adapted by the authors

Markov Decision Process (MDP)



University
of Exeter

An **MDP** models sequential decision making between an agent and an environment.

At each time step t :

- The agent observes the **current state** $s_t \in S$
 - The agent chooses an **action** $a_t \in A$
 - The environment returns a **reward** $r_t = r(s_t, a_t)$
 - The environment transitions to a **new state** s_{t+1}
- $$s_t \rightarrow a_t \rightarrow r_t, s_{t+1}$$

Markov Property

- The next state depends only on the current state and action, and not on earlier states or actions.

$$P(s_{t+1} | s_t, a_t)$$

Components of an MDP

An MDP is defined by: (S, A, P, R, γ)

- S : set of states, A : set of actions, P : transition probabilities, R : reward function and γ : discount factor

Learning Objective in an MDP

Learning Task

- The goal of reinforcement learning is to learn a **policy**

$$\pi : S \rightarrow A$$

which determines the action to take in each state.

Objective

- Find a policy that **maximizes cumulative reward over time**.
- The expected return or the value function from state s_t is:

$$V^\pi(s) = \mathbb{E} \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i+1} \mid s_t = s, \pi \right]$$

- The expectation averages over possible future outcomes (due to stochastic transitions or rewards)

Discount Factor: $\gamma \in [0,1]$ controls how future rewards are valued.

- $\gamma = 0 \rightarrow$ only immediate rewards matter
- $\gamma \approx 1 \rightarrow$ future rewards become important

Key insight: Good policies lead to states with higher long-term value.

Optimal Policy

Goal of RL

- Learn the best policy that maximises expected return.

Optimal policy

$$\pi^* = \arg \max_{\pi} V^{\pi}(s)$$

- $V^{\pi}(s)$: value of state s policy π
- π^* : policy that achieves the **maximum expected value**

Why learning is difficult

- The environment dynamics are unknown
- The agent only observes rewards through interaction
- The agent must learn from trial and error

Key insight

- Instead of directly learning the policy, we learn an **evaluation function** (value or action-value) to guide decisions.

Value Functions: Gridworld Example

To understand state values, consider a simple environment.

Gridworld setup

- 6 grid squares = 6 possible states
- Arrows = possible actions
- Numbers on arrows = immediate rewards $r(s, a)$

Reward Structure

- Entering the goal state G gives reward 100, otherwise reward is 0.

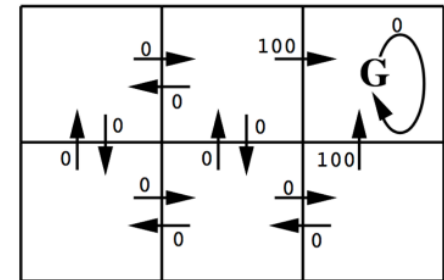
Terminal State

- Once the agent reaches G, it remains there.

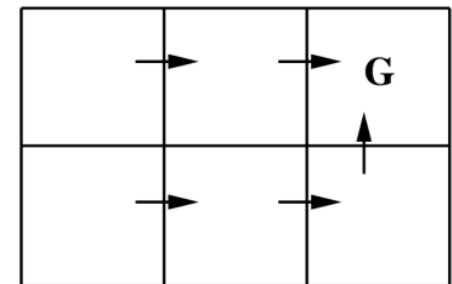
Optimal policy intuition

- An optimal policy moves the agent **towards the goal state**.

$r(s, a)$ values

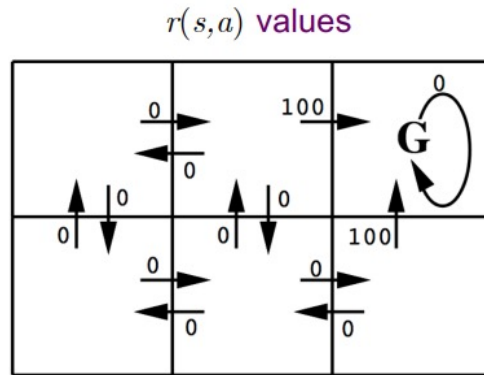


One optimal policy

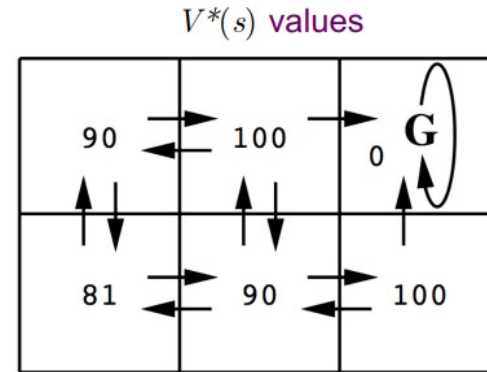


Computing State Values

Assume discount factor: $\gamma = 0.9$



Immediate rewards



Optimal state values

Example calculation

Consider the bottom-center state, the agent moves right toward the goal.

- Reward sequence: $0 \rightarrow 100 \rightarrow 0 \rightarrow \dots$

$$V^*(s) = 0 + \gamma 100 + \gamma^2 0 + \dots = 90$$

- In this gridworld the transitions are **deterministic**, so the expectation reduces to a single trajectory.
- States closer to the goal have higher value.

The Decision Problem

Which direction should the agent take?

- The agent starts from state A and must decide which direction to move.
- Each move leads to a different next state and possible future rewards.

Example Grid

- Each cell represents a state
- The green cell is the goal state
- Actions = Up, Down, Left, Right
- Reaching the goal gives a positive reward
- All other transitions have a reward 0

A		

Goal of the agent

- The agent wants to choose the direction that leads to the largest long-term reward.

Key Question: Which next state will lead to the best long-term outcome?

- This requires estimating the value of each state.

Comparing State Values

Choosing the Best Action

The agent should prefer state s_1 over s_2 whenever $V(s_1) > V(s_2)$

- where $V(s)$ represents the expected cumulative reward starting from state s .

Interpretation

- If moving to s_1 leads to higher future reward than s_2 , the agent should choose the action that leads to s_1 .

Challenge

- The agent cannot see the entire future path. It only sees the next state.

Key question: How can it estimate the long-term value of a state?

?		
A	?	

This leads to the **Bellman Equation**.

Bellman Equation

Bellman Equation: The value of a state is the best value achievable over all possible actions.

$$V(s) = \max_a [r(s, a) + \gamma V(s')]$$

- where a = action taken in state s , $r(s, a)$ is immediate reward after taking action a ; s' is next state reached from s and γ is discount factor

For each possible action, the agent chooses the action that maximises the total value.

Grid example with an important constraint

- The state directly below the goal (yellow cell) is one step away from the goal. $V(\text{yellow}) = 1$

1		
*		

Key Idea

- The Bellman equation evaluates all actions and chooses the one that leads to the highest long-term reward.

Applying the Bellman Equation

Choosing the Best Action

- The agent must choose the **best action from this state**.
- Assume: $\gamma = 0.9$, the Goal-adjacent state value: $V = 1$ and the immediate reward is 0

Example state

- From this state the agent has several actions:

Action	Next state value	Result
Up	1	$0 + 0.9 \times 1 = 0.9$
Right	0	$0 + 0.9 \times 0 = 0$
Left	0	$0 + 0.9 \times 0 = 0$

Bellman Decision Rule:

$$V(s) = \max(0.9, 0, 0) = 0.9$$

- The optimal action is **Up**, because it leads to the highest future value.

Value propagation

- $V(s) = \max_a(0 + 0.9 * 1) = 0.9$
- $V(s) = \max_a(0 + 0.9 * 0.9) = 0.81$
- $V(s) = \max_a(0 + 0.9 * 81) = 0.729$

1		
0.9		
0.81	0.729	

Part IV: Q Learning

Q-function

Action-Value Function (Q-function)

- Instead of estimating the value of a state, we estimate the value of a state-action pair.

$$Q(s, a)$$

- The expected cumulative reward obtained by taking action a in state s , and following the optimal policy afterward.

Bellman Equation for Q-values

- The action-value function satisfies a recursive relationship:

$$Q(s, a) = r(s, a) + \gamma \max_{a'} Q(s', a')$$

- s' = next state reached from s and a' = possible actions in the next state

Decision Rule

- Once Q-values are known, the optimal action is: $\pi(s) = \arg \max_a Q(s, a)$
- The agent simply chooses the action with the highest Q-value.

Key Challenge

- The transition dynamics are unknown and the reward function is unknown

Therefore, the agent must **learn Q-values through interaction**.

Temporal Difference (TD)

Learning from Experience

When the agent interacts with the environment it observes: (s, a, r, s')

From the Bellman equation, the desired value should be:

$$r(s, a) + \gamma \max_{a'} Q(s', a')$$

Temporal Difference Error

$$TD(s, a) = r(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

- This measures the difference between the target and the current estimate.
- The TD error tells us how much the Q-value should change.
- If $TD > 0$, the action was better than expected
- If $TD < 0$, the action was worse than expected

↑ $Q(s, a)$		

Q Learning

Updating Q-values

- Q-learning updates the action-value estimate using the TD error.

$$Q_t(s, a) = Q_{t-1}(s, a) + \alpha TD(s, a)$$

- Substituting the TD error gives:

$$Q_t(s, a) = Q_{t-1}(s, a) + \alpha \left[r(s, a) + \gamma \max_{a'} Q(s', a') - Q_{t-1}(s, a) \right]$$

Parameters

- Learning rate α : controls how quickly the agent updates its knowledge.
- Discount factor γ controls how important future rewards are.

Result

- Over time, the Q-values converge to the **optimal action values**, allowing the agent to follow:

$$\pi(s) = \arg \max_a Q(s, a)$$

Example Application

- Reinforcement learning applied to [Atari games \(e.g., Breakout\)](#).

Q Learning Example

Example Gridworld

- We illustrate Q-learning using a simple grid environment.
- The agent starts at state 1
- Each state corresponds to a location in the grid
- The agent can move North, South, East, or West

Initial Q-table: all Q-values start at 0.

	1	2	3	4	5	6
N	0	0	0	0	0	0
S	0	0	0	0	0	0
W	0	0	0	0	0	0
E	0	0	0	0	0	0

(-10 reward) 4	(0 reward) 5	Goal (+10 reward) 6
Starting position 1	(0 reward) 2	(0 reward) 3

First episode:

The agent experiences the trajectory: $1 \rightarrow 4 \rightarrow 5 \rightarrow 6$

- We will update the Q-values using

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- $\alpha = 0.8$ and $\gamma = 0.5$

Updating Q-values

- We now update the Q-values using the observed transitions.

Step 1: transition (1 → 4)

- Reward: $r = -10$
- Update: $Q(1, N) = 0 + 0.8(-10 + 0.5 \cdot 0 - 0) = -8$

Step 2: transition (4 → 5)

- Reward: $r = 0$
- $Q(4, E) = 0 + 0.8(0 + 0.5 \cdot 0 - 0) = 0$

Step 3: transition (5 → 6)

- Reward: $r = 10$
- $Q(5, E) = 0 + 0.8(10 + 0.5 \cdot 0 - 0) = 8$

(-10 reward) 4	(0 reward) 5	Goal (+10 reward) 6
Starting position 1	(0 reward) 2	(0 reward) 3

Result

- The Q-table now begins to store useful information about which actions lead to higher rewards.
- Over many episodes, these Q-values converge toward the optimal policy.

Summary

Core Components of RL

- State (s) — the current situation of the agent
- Action (a) — choices the agent can take
- Reward (r) — feedback from the environment
- Policy (π) — strategy that maps states to actions
- Value (v) – Expected long-term return with discount, as opposed to short-term reward

Key Challenges

- Exploration vs Exploitation
 - The agent must balance trying new actions and using known good ones.
- Sequential Decisions
 - Actions influence future states and rewards.

Problem Formulation

- Multi-armed Bandits — learning with independent decisions
- Markov Decision Processes (MDP) — sequential decision making

Value-Based Learning

- Value function estimates long-term reward
- Bellman equation expresses recursive value relationships

Learning from Experience

- Temporal Difference (TD) learning updates values from experience
- Q-learning learns action values